

Efficient Machine Learning Models for Intrusion Detection on ESP32 Microcontroller

GODSPower OGAGA UTI* and HENRIK STEEN*, Østfold University of Applied Sciences, Norway

The Internet of Things continues to grow rapidly, and many of the resulting devices communicate directly over cellular or non-Ethernet links that bypass the centralised firewalls and Intrusion Detection Systems (IDSs) that traditionally protect them. Running an IDS locally on the device itself is therefore attractive, but typical IoT microcontrollers offer only hundreds of kilobytes of memory and clock speeds in the low hundreds of MHz, which is well below what conventional ML-based IDSs require. This paper examines whether IDS inference can run on IoT, how much test data can be fit on-device, and which metrics can be measured locally. In this paper, we deployed four models trained on the TON_IoT network dataset to a commodity ESP32 microcontroller. The models include a Decision Tree and a LightGBM model for multi-class attack detection, and a Feedforward Neural Network and a Support Vector Machine classifier for binary detection. Each model is evaluated on 7,360 test samples embedded directly in flash, at CPU clocks of 240, 160, 80, and 40 MHz. The Decision Tree reached 99.14% multi-class accuracy at up to 68,050 inferences per second, the binary Neural Network reached 99.92%, and classification quality stayed constant as the clock dropped. The results show that competitive on-device IDS is achievable on a low-cost microcontroller well below peak clock speed, leaving headroom and energy for the device's primary task.

CCS Concepts: • **Computer systems organization** → **Embedded software**; • **Security and privacy** → *Intrusion detection systems*.

Additional Key Words and Phrases: TinyML, Cybersecurity, Internet of Things (IoT), Industrial Internet of Things (IIoT), intrusion detection systems (IDSs), TON_IoT dataset.

ACM Reference Format:

Godspower Ogaga Uti and Henrik Steen. 2026. Efficient Machine Learning Models for Intrusion Detection on ESP32 Microcontroller. 1, 1 (June 2026), 10 pages. <https://orcid.org/0009-0006-1817-7789>

1 Introduction

The number of connected IoT devices continues to grow at an accelerating pace, with estimates being roughly 55.7 billion devices online by 2025 [Hojlo 2021]. Many of these devices communicate directly over cellular or non-Ethernet links and therefore bypass the centralised routers and firewalls where traditional Intrusion Detection Systems are deployed. At the same time, the microcontrollers that typically power such devices have only hundreds of kilobytes of SRAM (for data and instruction) and clock speeds in the low hundreds of MHz, which rules out conventional machine learning-based IDS that rely on large models and high-performance CPUs.

This paper addresses three concrete problems that follow from this mismatch. First, can a regular IoT microcontroller execute IDS inference entirely on-device for both *multi-class* attack detection (distinguishing between specific attack families) and *binary* detection (benign versus malicious), without offloading to the cloud? Second, how much of the labelled test data can be embedded into the same device alongside the model so that the

*Both authors contributed equally to this research.

Authors' Contact Information: Godspower Ogaga Uti, godspowu@hiof.no; Henrik Steen, henrik.steen@hiof.no, Østfold University of Applied Sciences, Halden, Norway.

2026. ACM XXXX-XXXX/2026/6-ART
<https://orcid.org/0009-0006-1817-7789>

evaluation is closer to a real-world situation? Third, what performance and efficiency metrics can be collected directly from the device across a range of CPU frequencies, so that the trade-off between throughput and energy can be characterized empirically? We target the ESP32 as a representative low-cost microcontroller and use the TON_IoT Telemetry dataset [Booij et al. 2022; Moustafa 2021; Moustafa et al. 2020], which is widely adopted as a benchmark for realistic IoT intrusion data.

The most directly related work is by Lundqvist et al. [Lundqvist et al. 2025], who evaluated four lightweight models (Decision Tree, LightGBM, a Feedforward Neural Network, and TinyML FNN) on the TON_IoT dataset using a Raspberry Pi 4 whose cores and clock speed were throttled to approximate IoT-class hardware. They reported an F1-score of 0.976 and an estimated throughput of around 120,000 packets per second for their TinyML FNN, and identified a clear research gap: their evaluation relied on a Linux host running CPython rather than a real microcontroller, and they explicitly proposed deployment on actual IoT microcontrollers as future work. Our study follows this recommendation by taking the same dataset and model families and running them on an ESP32 instead of an emulated environment.

Broader reviews of ML-based IDS for IoT confirm the shift away from centralized detection toward on-device approaches. Thakkar and Lohiya [Thakkar and Lohiya 2021] surveyed IDS architectures for IoT and emphasized the need for resource-efficient, distributed detection. Verma and Ranga [Verma and Ranga 2020] compared classifiers such as CART and XGBoost on Raspberry Pi hardware and showed that certain tree-based models are well suited to on-device intrusion detection, while Amouri et al. [Amouri et al. 2020] built a two-stage Random Forest and regression system for mobile IoT. Javed et al. [Javed et al. 2024] went further and demonstrated a TinyML-based IDS running locally on a smart thermostat for binary detection, showing the practical feasibility of on-device deployment.

The enabling technologies for this shift are TinyML and related embedded inference frameworks. TinyML uses quantization, pruning, and compact architectures to fit neural networks into devices with only a few hundred kilobytes of memory [Alajlan and Ibrahim 2022; Dutta and Bharali 2021]. On the software side, TensorFlow Lite for Microcontrollers [Warden and Situnayake 2019] allows quantized neural networks to be exported as C arrays and run on Arm Cortex-M and Xtensa class devices, while the emlearn library [Nordby 2019] converts models into C99-compliant code suitable for the same platforms. Together these tools let us deploy a set of ML models to the ESP32 without exceeding its limited resources.

Although these works individually move the field toward on-device IDS, each of them leaves at least one practical question unanswered. Lundqvist et al. [Lundqvist et al. 2025] ran their full pipeline on a Raspberry Pi 4 with throttled cores and clock speed, but never executed the converted models on a real microcontroller and only estimated microcontroller performance through a derived packets-per-MHz formula. Thakkar and Lohiya [Thakkar and Lohiya 2021] provide a survey of IDS strategies for IoT but do not implement or measure any model on constrained hardware themselves. Verma and Ranga [Verma and Ranga 2020] evaluate classifiers on a Raspberry Pi 3, which is still substantially more capable than typical microcontrollers and therefore does not reflect deployment on low-end IoT devices. Amouri et al. [Amouri et al. 2020] focus on detection accuracy in a simulated mobile IoT setup and do not report model size, memory use, or throughput on resource-constrained hardware. Javed et al. [Javed et al. 2024] demonstrate on-device TinyML IDS on a smart thermostat but only for binary classification and a single hardware target, with no comparison across model families or CPU frequencies. The TinyML overviews by Alajlan and Ibrahim [Alajlan and Ibrahim 2022] and Dutta and Bharali [Dutta and Bharali 2021] characterize the design space at a high level rather than evaluating IDS specifically. The TensorFlow Lite for Microcontrollers reference [Warden and Situnayake 2019] and the emlearn toolchain [Nordby 2019] describe enabling technology rather than IDS evaluation results. The TON_IoT references [Booij et al. 2022;

Moustafa 2021; Moustafa et al. 2020] and the IDC market estimate [Hojlo 2021] provide data and motivation but do not themselves evaluate on-device detection.

This paper addresses those limitations by implementing and measuring four trained models, a Decision Tree and a LightGBM classifier for multi-class detection and a Feedforward Neural Network and a Support Vector Machine classifier for binary detection, on the ESP32 itself rather than on a throttled Linux host. The Decision Tree and LightGBM are converted to C using emlearn and TL2cgen, the Neural Network is exported through TensorFlow Lite for Microcontrollers, and the SVM is converted using m2cgen. We embed 7,360 labelled TON_IoT test samples directly in flash, run inference at four CPU frequencies (240, 160, 80, 40 MHz), and collect classification metrics together with inferences per second and inferences per MHz for every run. This setup turns previously estimated microcontroller performance into measured numbers, covers both detection tasks on a single device, and gives a direct comparison across four model families on identical hardware.

2 Dataset

The training and evaluation of all four models is based on the TON_IoT Train Test Network dataset, originally produced at the UNSW Canberra Cyber testbed [Moustafa 2021; Moustafa et al. 2020]. The testbed was designed to replicate realistic Industry 4.0 and Industrial IoT architectures, generating Telemetry data, Operating-system data, and Network data under both benign and attack conditions. Among the dataset variants released by the authors, the Train Test Network subset is the network-flow split intended for benchmarking machine learning-based network intrusion detection, and is widely used in follow-up works such as Booij et al. [Booij et al. 2022].

The Train Test Network subset contains 211,043 labelled network flows with 44 raw flow-level features. Two label fields are available: a binary label (benign versus malicious) and a multi-class type field that distinguishes between nine attack families plus the normal class. The attack families are Scanning, Distributed Denial of Service (DDoS), Denial of Service (DoS), Backdoor, Ransomware, Cross-Site Scripting (XSS), Password cracking, Data injection, and Man-in-the-Middle (MITM). The class distribution of the original dataset is shown in Table 1. Most attack categories contribute 20,000 flows each, the normal class contributes 50,000 flows, and MITM is heavily underrepresented at 1,043 flows. The mix of high-volume flooding attacks (DoS, DDoS, scanning), application-layer attacks (XSS, injection, password), and stealthier protocol attacks (MITM, backdoor, ransomware) is a key reason this dataset is used as a standard benchmark for IDS work on IoT networks.

Class	Number of flows
Normal	50,000
Backdoor	20,000
DDoS	20,000
DoS	20,000
Injection	20,000
Password	20,000
Ransomware	20,000
Scanning	20,000
XSS	20,000
MITM	1,043
Total	211,043

Table 1. Class distribution of the TON IoT Train Test Network subset

For our experiments we follow standard practice for this dataset: leakage-prone identifiers (`dst_ip`, and the binary `label` when training the multi-class models) are removed, columns with a vast majority of NaN values are dropped, duplicates are removed, categorical fields are label-encoded, and the data is then split into an 80/20 train/test partition with stratified sampling so that minority classes such as MITM remain represented in both splits. One-hot encoding of categorical features is applied for `emlearn` compatability. The neural network additionally uses interquartile-range scaling for outliers. After preprocessing, each input flow is represented by 83 features, which is the input dimensionality embedded into the ESP32. Out of the 20% (36,800) test split, 7,360 samples were embedded as a binary array directly in device flash so that on-device inference does not depend on any host-side data delivery; this is the largest contiguous evaluation set the ESP32 could host together with the model and the runtime.

3 Proposed Solution

The implementation is split into three components: an offline training and conversion pipeline, a small set of Python scripts that turn the held-out test split into C header files, and a per-model PlatformIO project that compiles each converted model together with its embedded test data and runs it on the ESP32. The target hardware is the ESP32, a dual-core Tensilica Xtensa microcontroller with 520 KB of SRAM and up to 4 MB of embedded flash, selected because it is widely deployed in IoT products.

3.1 Training and conversion

First the TON_IoT dataset is loaded, then preprocessing is applied as described in Section 2, and four models are trained with either multi-class targets, or a binary classification target. Scikit-learn is utilized for the Decision Tree and SVM Classifier, LightGBM for the gradient-boosted ensemble, and Keras/TensorFlow for the Feedforward Neural Network. The same 83-feature, train/test-split view of the data is used for every model, which keeps the comparison consistent across model families. We chose a Decision Tree and a LightGBM model to cover the multi-class task because they represent two very different trade-offs in tree-based learning (a single small tree versus a gradient-boosted ensemble), and we paired them with a Neural Network and a SVM classifier for binary detection to cover both a modern TinyML neural workload and a classical kernel-based baseline.

No single conversion library covers all four model families well, so each model is exported with the tool that fits it best. The Decision Tree is converted with `emlearn` [Nordby 2019] using `method='inline'`, which inlines the entire tree as nested `if/else` branches into a self-contained C99 header. This produces extremely small, branch-only code that the Xtensa core can run without any external runtime. `emlearn` did not scale to our LightGBM ensemble because the resulting model grew too large for the ESP32 toolchain to compile, so we use `Treelite` together with `TL2cgen` instead; `TL2cgen` produces a `predict` function in C, and the resulting model is small enough to fit the ESP. The Feedforward Neural Network is converted with the standard TensorFlow Lite converter into a `.tflite` file and executed at runtime by TensorFlow Lite for Microcontrollers [Warden and Situnayake 2019], which provides a small interpreter and a curated set of operator kernels that fit on the ESP32. Finally, the SVM classifier is converted with `m2cgen`, which emits a stand-alone C function that computes the linear decision directly from the model coefficients.

3.2 Embedding the test data

For the on-device evaluation to reflect the device itself rather than the serial port that feeds it, the held-out test split has to live in flash next to the model. The test data scripts reload the same dataset used by the models and is preprocessed in the same fashion.

The test data scripts emit the features and labels as a C header, with an array containing the raw bytes of the matrix. Storing the matrix as a packed byte array, instead of as a textual literal of floating-point numbers, is what allows 7,360 samples of 83 features to fit alongside the model in the ESP32's flash without overflowing the source-code parser or the compiler's RAM during the build. The byte-level representation also makes the on-device read path a simple cast, which keeps the inference loop's measurement free of formatting overhead.

3.3 Deploying the models

Each model has its own PlatformIO project, targeting the feather ESP32 board with the Arduino framework on top of the Espressif platform. These projects all use a custom huge_app partition table so that the application section is large enough to hold both the converted model and the embedded test data. The DT, LightGBM, and SVM projects include the converted model header and the test data directly. The Neural Network project instead embeds its model into the firmware image via PlatformIO's board_build.embed_files option and loads it at runtime through the TensorFlow Lite Micro interpreter.

The main file of every project follows the same pattern: it sets the CPU clock at either 240, 160, 80, or 40 MHz, prints the active frequency, iterates over all embedded samples, calls the model's predict function, and accumulates per-class true positive (TP), false positive (FP), and false negative (FN) counters. After the loop it computes accuracy, macro precision/recall/F1 for the multi-class models or binary precision/recall/F1 with a confusion matrix for the binary models, and reports total runtime, average inference time per sample, and inferences per second and per MHz over the serial monitor. The four log files used in Section 4 are direct captures of that serial output for the four CPU clocks.

4 Performance Evaluations

Across all four models and all four clock speeds, classification quality on the embedded test set was essentially independent of CPU frequency, as expected for deterministic inference code. The Decision Tree reached 99.14% accuracy and a macro F1 of 0.9813 for ten-class detection at 68,050.5 inferences per second on the full 240 MHz clock, and still delivered 26,287.5 inferences per second when the clock was dropped to 40 MHz. LightGBM reached 98.86% accuracy and a macro F1 of 0.9665 but was roughly two orders of magnitude slower, peaking at 990.3 inferences per second at 240 MHz. On binary detection, the Neural Network reached 99.92% accuracy with precision, recall, and F1 of 0.9995 at 6,108.4 inferences per second, while the Support Vector Machine classifier reached 99.89% accuracy and 0.9993 F1 at 16,614.6 inferences per second. Table 2 summarizes the peak and minimum throughput together with classification metrics for each model.

Model	Task	Acc.	F1	Inf/s @240	Inf/s @40
Decision Tree	Multi (10)	99.14%	0.9813	68,050	26,287
LightGBM	Multi (10)	98.86%	0.9665	990	321
Neural Network	Binary	99.92%	0.9995	6,108	1,390
SVM	Binary	99.89%	0.9993	16,614	2,744

Table 2. Summary of ESP32 results at 240 MHz and 40 MHz on 7,360 embedded test samples (83 features).

The remainder of this section breaks the same runs down per model, reporting every metric collected by the on-device logger across all four CPU frequencies: total runtime, average inference time per sample, throughput in inferences per second, an efficiency figure given as inferences per MHz, and the full classification metrics. Tables 3 through 7 contain the detailed per-model results.

4.1 Decision Tree – Multi-class

The Decision Tree was converted with emlearn and is the smallest and fastest of the four models. It correctly classified 7,297 of the 7,360 embedded samples in every run, giving an accuracy of 99.14% and a macro F1 of 0.9813 that is independent of the clock speed. Throughput stayed relatively consistent until clockspeed was down to 80 MHz, and from there scaled almost linearly with frequency. Inferences per second ranged from 68,050.5 at 240 MHz down to 26,287.5 at 40 MHz; even the slowest configuration processed all 7,360 samples in 310 ms. The efficiency metric (inferences per MHz) actually increases at lower clocks, indicating that the Decision Tree is bottlenecked by raw clock cycles rather than memory bandwidth and therefore extracts more work per MHz when the clock is throttled.

Table 3. Decision Tree multi-class classification: detailed ESP32 results. * stands for identical number

Metric	240 MHz	160 MHz	80 MHz	40 MHz
Total runtime (ms)	114	123	151	310
Avg. time/sample (μ s)	14.69	15.66	18.56	38.04
Inferences/sec	68,050.5	63,865.1	53,869.3	26,287.5
Inferences/MHz	283.54	399.16	673.37	657.19
Accuracy	99.14%	*	*	*
Macro Precision	0.9796	*	*	*
Macro Recall	0.9831	*	*	*
Macro F1 Score	0.9813	*	*	*

4.2 LightGBM – Multi-class

The LightGBM model was converted with TL2cgen and is by far the slowest of the four. Although it achieves good multi-class metrics (98.86% accuracy, 0.9665 macro F1, identical for all clock settings), each inference takes between 1,010 and 3,112 μ s, two orders of magnitude longer than the Decision Tree. Throughput drops from 990.3 inferences per second at 240 MHz to only 321.3 at 40 MHz, where evaluating all 7,360 samples takes more than 24 seconds. Inferences per MHz are correspondingly low (4–8), confirming that the gradient-boosted ensemble does not fit the throughput budget of this microcontroller as well as a single tree does.

Table 4. LightGBM multi-class classification: detailed ESP32 results. * stands for identical number

Metric	240 MHz	160 MHz	80 MHz	40 MHz
Total runtime (ms)	7,834	8,841	11,911	24,391
Avg. time/sample (μ s)	1,009.81	1,135.66	1,519.43	3,112.12
Inferences/sec	990.3	880.5	658.1	321.3
Inferences/MHz	4.13	5.50	8.23	8.03
Accuracy	98.86%	*	*	*
Macro Precision	0.9559	*	*	*
Macro Recall	0.9820	*	*	*
Macro F1 Score	0.9665	*	*	*

4.3 Neural Network – Binary

The Feedforward Neural Network was quantized and exported with TensorFlow Lite for Microcontrollers. It misclassified only 6 of 7,360 samples (3 false positives and 3 false negatives), giving an accuracy of 99.92% and a precision, recall, and F1 score of 0.9995. Inference time scales from 163.71 μ s at 240 MHz to 719.46 μ s at 40 MHz, with throughput dropping from 6,108.4 to 1,389.9 inferences per second. Efficiency (inferences per MHz) improves as the clock is reduced, showing that the neural network is compute-bound rather than memory-bound on this device.

Table 5. Neural Network binary classification: detailed ESP32 results. * stands for identical number

Metric	240 MHz	160 MHz	80 MHz	40 MHz
Total runtime (ms)	1,400	1,757	2,851	5,844
Avg. time/sample (μ s)	163.71	209.93	350.99	719.46
Inferences/sec	6,108.4	4,763.6	2,849.1	1,389.9
Inferences/MHz	25.45	29.77	35.61	34.75
Accuracy	99.92%	*	*	*
Precision	0.9995	*	*	*
Recall	0.9995	*	*	*
F1 Score	0.9995	*	*	*

		True labels	
		Positive	Negative
Predicted labels	Positive	5729	3
	Negative	3	1625

Table 6. Neural Network binary classification: confusion matrix.

4.4 SVM – Binary

The Support Vector Machine classifier was converted with m2cgen and represents a classical kernel-based baseline. It misclassified 8 of 7,360 samples (4 false positives and 4 false negatives), reaching 99.89% accuracy and a precision, recall, and F1 of 0.9993. The SVM sits between the Decision Tree and the Neural Network on throughput: 16,614.6 inferences per second at 240 MHz down to 2,743.5 at 40 MHz, with efficiency stable around 68–70 inferences per MHz across all clocks. This is roughly three times faster than the Neural Network at every clock speed for binary detection, at the cost of a slightly higher error count (8 versus 6 misclassifications out of 7,360).

Table 7. SVM binary classification: detailed ESP32 results. * stands for identical number

Metric	240 MHz	160 MHz	80 MHz	40 MHz
Total runtime (ms)	757	1,051	1,946	3,986
Avg. time/sample (μ s)	60.19	89.30	177.98	364.50
Inferences/sec	16,614.6	11,198.8	5,618.6	2,743.5
Inferences/MHz	69.23	69.99	70.23	68.59
Accuracy	99.89%	*	*	*
Precision	0.9993	*	*	*
Recall	0.9993	*	*	*
F1 Score	0.9993	*	*	*

		True labels	
		Positive	Negative
Predicted labels	Positive	5728	4
	Negative	4	1624

Table 8. SVM binary classification: confusion matrix.

4.5 Comparison with Existing Works

Direct numerical comparison is only meaningful with works that use the same TON_IoT Train Test Network subset and report multi-class metrics, since our binary models cannot be aligned one-to-one with multi-class baselines. Lundqvist et al. [Lundqvist et al. 2025] report multi-class numbers for the same four model families on a Raspberry Pi 4. Booi et al. [Booi et al. 2022] train a Random Forest on the same dataset, and Hajla et al. (cited as a Voting baseline in [Lundqvist et al. 2025]) report ensemble multi-class numbers. Table 9 compares our ESP32 multi-class results to these published values.

Table 9. Multi-class comparison on the TON IoT Train Test Network dataset.

Source	Model	Accuracy	Macro F1
This paper (ESP32)	Decision Tree	99.14%	0.9813
This paper (ESP32)	LightGBM	98.86%	0.9665
Lundqvist et al. (Raspberry Pi 4) [Lundqvist et al. 2025]	Decision Tree	98.7%	0.987
Lundqvist et al. (Raspberry Pi 4)	LightGBM	97.6%	0.976
Lundqvist et al. (Raspberry Pi 4)	FNN	97.4%	0.976
Lundqvist et al. (Raspberry Pi 4)	TinyML FNN	97.4%	0.976
Booij et al. [Booij et al. 2022]	Random Forest	98.1%	0.973
Voting ensemble [Lundqvist et al. 2025]	Voting	96.3%	0.886

5 Discussion

There are two vital observations from Table 9. First, the multi-class accuracy and macro F1 we measured on the ESP32 are better than those reported by Lundqvist et al. [Lundqvist et al. 2025] for the same model families on a much more powerful Raspberry Pi 4. The Decision Tree differs by less than half a percentage point in accuracy and a few thousandths in F1, and our LightGBM result is improved compared to their LightGBM and TinyML FNN. Moving from a throttled Linux host to a real microcontroller therefore did not cost us classification quality, which is a non-trivial result given the difference in runtime, memory, and instruction set. Second, our Decision Tree comfortably outperforms the Random Forest baseline of Booij et al. [Booij et al. 2022] and the Voting ensemble reported in [Lundqvist et al. 2025] on macro F1, while running on hardware that costs roughly an order of magnitude less than a Raspberry Pi 4. Combined with the binary results, where both the Neural Network and the SVM classifier exceed 99.89% accuracy on-device, this shows that the ESP32 can host competitive multi-class *and* binary IDS, which is the gap left open by the related work discussed in the Introduction.

5.1 Statement of Broader Impacts

Prior work relied on throttled host measurements, but this paper decisively presents empirical evidence from real hardware. It establishes a practical alternative to the computational demands of conventional ML-based IDS. The results confirm that the ESP32 can efficiently run an IDS at a frequency well below its peak clock, leaving substantial headroom and energy margin for the primary application on the same device.

6 Conclusion

In this study, we have successfully deployed a series of lightweight ML-based IDS directly on a resource-constrained, low-cost ESP32 microcontroller. Using a dataset (TON_IoT) designed to reflect a realistic IoT network testbed, our IDS detected and identified security threats at different clock frequencies with minimal operational impact on the device itself. Additionally, The comparison across four models reveals that the optimal choice depends on model family, not a trade-off between accuracy and deployability. The Decision Tree excels in throughput; the Neural Network and SVM lead in binary accuracy; and LightGBM, though accurate, is unsuitable for this hardware due to its slow performance at 40 MHz. Overall this paper proves that IDS inference can run well on a commonly used IoT device, which provides an alternative to offloading these types of computation to the cloud.

Acknowledgments

To Thuy Nga Dinh, for the mentorship.

References

- Norah N. Alajlan and Dina M. Ibrahim. 2022. TinyML: Enabling of Inference Deep Learning Models on Ultra-Low-Power IoT Edge Devices for AI Applications. *Micromachines* 13, 6 (2022), 851. doi:10.3390/mi13060851
- Amar Amouri, Vishwa T. Alaparthi, and Salvatore D. Morgera. 2020. A Machine Learning Based Intrusion Detection System for Mobile Internet of Things. *Sensors* 20, 2 (2020), 461. doi:10.3390/s20020461
- Tim M. Booi, Irina Chiscop, Erik Meeuwissen, Nour Moustafa, and Frank T. H. den Hartog. 2022. ToN_IoT: The Role of Heterogeneity and the Need for Standardization of Features and Attack Types in IoT Network Intrusion Data Sets. *IEEE Internet of Things Journal* 9, 1 (2022), 485–496. doi:10.1109/JIOT.2021.3085194
- Diptaa Lakshmi Dutta and Sudipta Bharali. 2021. TinyML Meets IoT: A Comprehensive Survey. *Internet of Things* 16 (2021), 100461. doi:10.1016/j.iot.2021.100461
- Jonathan Hojlo. 2021. Future of Industry Ecosystems: Shared Data and Insights. International Data Corporation (IDC). <https://blogs.idc.com/2021/01/06/future-of-industry-ecosystems-shared-data-and-insights/>
- Abbas Javed, Amna Ehtsham, Muhammad Jawad, Muhammad Naeem Awais, Ayyaz-ul-Haq Qureshi, and Hadi Larijani. 2024. Implementation of Lightweight Machine Learning-Based Intrusion Detection System on IoT Devices of Smart Homes. *Future Internet* 16, 6 (2024), 200. doi:10.3390/fi16060200
- Jonathan Lundqvist, Anel Hadzic, Torstein Mo Kirkeluten, Håkon Pedersen, Jacob Holth, Magnus H. Johansson, and Moritz P. N. Halkjelsvik. 2025. Lightweight Machine Learning Models for Intrusion Detection on IoT Devices. In *Norsk IKT-konferanse for forskning og utdanning (NIKT)*.
- Nour Moustafa. 2021. A New Distributed Architecture for Evaluating AI-based Security Systems at the Edge: Network TON_IoT Datasets. *Sustainable Cities and Society* 72 (2021), 102994. doi:10.1016/j.scs.2021.102994
- Nour Moustafa, Marwa Keshky, Essam Debiez, and Helge Janicke. 2020. Federated TON_IoT Windows Datasets for Evaluating AI-Based Security Applications. In *Proceedings of the 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, 848–855. doi:10.1109/TrustCom50675.2020.00114
- Jon Nordby. 2019. emlearn: Machine Learning Inference Engine for Microcontrollers and Embedded Devices. Zenodo. doi:10.5281/zenodo.2589394
- Ankit Thakkar and Ritika Lohiya. 2021. A Review on Machine Learning and Deep Learning Perspectives of IDS for IoT: Recent Updates, Security Issues, and Challenges. *Archives of Computational Methods in Engineering* 28, 4 (2021), 3211–3243. doi:10.1007/s11831-020-09496-0
- Abhishek Verma and Virender Ranga. 2020. Machine Learning Based Intrusion Detection Systems for IoT Applications. *Wireless Personal Communications* 111, 4 (2020), 2287–2310. doi:10.1007/s11277-019-06986-8
- Pete Warden and Daniel Situnayake. 2019. *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media.

Received 2 May 2026; revised 12 March 2026; accepted 13 May 2026